Project Plan

Two-Phase Deep Learning Project

Phase 1: Cross-Lingual Textual Entailment for Hindi Using Transformer-Based Deep Learning

Phase 2: Technical Review Paper on Knowledge Graph Construction for Indic Languages

Deep Learning Course Assignment + Undergraduate Publication

Prepared for: Professor

Prepared by: Mohammed Ufraan

July 20, 2026

Contents

1 Executive Summary **1**

2 Background **2**
2.1 The Original Paper (Khader et al., 2016) . . . 2
2.2 Limitations of the Original Work . . . 2

3 Proposed Approach **3**
3.1 Overview . . . 3
3.2 Why XLM-RoBERTa? . . . 3
3.3 Why XNLI Dataset? . . . 3

4 Prerequisites **4**
4.1 Python Programming . . . 4
4.2 Mathematics Fundamentals . . . 4
4.3 Neural Networks & Deep Learning . . . 4
4.4 The Transformer Architecture . . . 5
4.4.1 Embedding Layer . . . 5
4.4.2 Self-Attention Mechanism . . . 5
4.4.3 Multi-Head Attention . . . 5
4.4.4 Transformer Block Structure . . . 6
4.4.5 Classification Head . . . 6
4.4.6 XLM-RoBERTa vs. BERT . . . 6
4.5 Tokenization in Detail . . . 6
4.6 Transfer Learning & Fine-Tuning . . . 7
4.7 The Textual Entailment (NLI) Task . . . 7
4.7.1 Task Structure . . . 7
4.7.2 Why 3 classes instead of 2 . . . 7
4.8 PyTorch for This Project . . . 8
4.9 Hugging Face Ecosystem . . . 8
4.10 Evaluation Metrics . . . 8
4.10.1 Accuracy . . . 8
4.10.2 Precision, Recall, F1 . . . 9
4.10.3 Confusion Matrix . . . 9
4.11 Hardware Awareness . . . 9
4.12 Book Reading Plan: “Fundamentals of Deep Learning” (Buduma & Locascio) . . . 10
4.13 Additional Learning Resources . . . 10

5 Technical Stack **12**

I Phase 1: NLI Assignment **13**

6 Implementation Plan **14**
6.1 Phase 1: Environment Setup (Day 1) . . . 14
6.2 Phase 2: Dataset Exploration (Day 2) . . . 14
6.3 Phase 3: Model & Tokenizer Setup (Day 3) . . . 14
6.4 Phase 4: Training Pipeline (Day 4-5) . . . 14
6.5 Phase 5: Evaluation (Day 6) . . . 14
6.6 Phase 6: Inference Demo (Day 7) . . . 14

7 Expected Results & Comparison **15**

II Phase 2: KG Survey Paper **16**

8 Overview **17**

9 Why a Review Paper? **17**

10 Template Paper **17**

11 Research Methodology **17**

12 Expected Timeline **18**

13 Target Venues **18**

14 Paper Structure (Phase 1: NLI Assignment) **19**

15 Paper Structure (Phase 2: KG Survey) **19**

16 Timeline (8 Weeks) **20**
16.1 Phase 1: NLI Assignment (Weeks 1–3) . . . 20
16.2 Phase 2: KG Survey Paper (Weeks 4–8) . . . 20

17 Deliverables **21**
17.1 Phase 1 (NLI Assignment) . . . 21
17.2 Phase 2 (KG Survey Paper) . . . 21

18 References **22**

Executive Summary

This document outlines a **two-phase plan** spanning 8 weeks.

Phase 1 (Weeks 1–3): Adapt the research paper *"Textual Entailment for Arabic Language Based on Lexical and Semantic Matching"* (Khader et al., 2016) for a Deep Learning course assignment. The original paper achieves 61% precision/recall on Arabic textual entailment using hand-crafted lexical features. We adapt it to Hindi using modern transformer-based deep learning (XLM-RoBERTa fine-tuned on XNLI), targeting 85%+ accuracy.

Phase 2 (Weeks 4–8): Write a technical review paper on **Knowledge Graph Construction for Hindi / Indic Languages**, following the structure of the Arabic KG survey (Ahmed et al., 2022) as a template. This will be a supervised undergraduate publication with guidance from the course professor.

Table 1: Assignment Options Satisfied (Phase 1)

Option	How This Project Satisfies It
1	Different language: Arabic → Hindi
2	Improved results: 61% → 85%+
3	Changed algorithm: Lexical rules → Transformers
4	Changed methodology: Threshold-based → Fine-tuning
5	Full implementation with runnable code
6	Different dataset: ArbTEDS → XNLI Hindi

Background

The Original Paper (Khader et al., 2016)

- **Task:** Recognizing Textual Entailment (RTE) for Arabic
- **Approach:** Lexical analysis (word overlap + bigram matching) + semantic matching (synonyms)
- **Dataset:** ArbTEDS — 618 Text-Hypothesis pairs
- **Results:** 61% precision and recall (Entails: 68% precision, NotEntails: 58% precision)
- **Tools:** Python, ISRI stemmer, MSTParser, Mada POS tagger

Limitations of the Original Work

- Manual feature engineering — does not scale to new domains
- Simple threshold-based decision — no learning involved
- Small dataset (618 pairs) — limits generalization
- Binary classification (Entails / NotEntails) — no neutral/contradiction distinction
- The authors themselves note in their future work that testing on other languages is needed

Proposed Approach

Overview

We will build a Textual Entailment system for Hindi by fine-tuning **XLM-RoBERTa**, a multilingual transformer pre-trained on 100 languages, on the **XNLI dataset**.

Why XLM-RoBERTa?

- Pre-trained on 100 languages including Hindi and Urdu
- State-of-the-art on cross-lingual NLP benchmarks
- Achieved +9.2% accuracy improvement for Urdu on XNLI over previous models (Conneau et al., 2019)
- No language identification needed — model infers language automatically
- Available in base (125M params) and large (355M params) variants

Why XNLI Dataset?

- 5,000 test pairs and 2,500 validation pairs per language
- Professionally translated from English MultiNLI by human translators
- Covers 14 diverse text genres (fiction, telephone speech, letters, 9/11 report, etc.)
- Three labels: Entailment, Neutral, Contradiction — richer than the original paper's binary setup
- Freely available on Hugging Face Datasets

Prerequisites

This section describes what you need to know — and what to study if you don't already. Each subsection explains a concept, why it matters for this project, and suggests concrete learning goals. Work through them in order.

Python Programming

You need comfort with Python beyond tutorials. Specifically:

- **Data structures:** lists, dicts, sets, list comprehensions — you will slice datasets, batch indices, and map functions over sequences.
- **Functions & classes:** the training script defines a custom `Log` class and a `tokenize_fn`; you should understand `self`, `__init__`, and how methods work.
- **pip & virtual environments:** create an isolated environment so library versions don't clash:

```
python3 -m venv .venv && source .venv/bin/activate && pip install ...
```

- **Reading tracebacks:** most errors during ML training come from shape mismatches, wrong dtypes, or missing keys — deciphering a Python traceback is a core skill.

If you're at "I've written a few scripts": you're fine. If you're at "I just started learning Python," watch a crash course first (e.g., Corey Schafer's Python tutorials on YouTube).

Mathematics Fundamentals

You do not need a math degree, but these three things appear constantly:

- **Matrices & vectors:** every batch of tokenized text is a matrix of shape `[batch_size, max_length]`. The transformer processes this matrix through linear algebra operations. Be comfortable with what a matrix is, what dimensions mean, and how matrix multiplication works.
- **Probabilities & logarithms:** the model outputs raw scores called *logits*; these are passed through `softmax` to convert them into probabilities (three numbers that sum to 1). The training loss is *cross-entropy*, which measures how far the predicted probability distribution is from the correct label. Lower cross-entropy = better prediction.
- **Gradients (conceptually):** training adjusts the model's weights by computing *gradients* — which direction and how much to tweak each weight to reduce the loss. PyTorch handles this automatically via `loss.backward()`, but understand that it's just calculus (chain rule) applied at scale.

Study target: watch 3Blue1Brown's "Essence of Linear Algebra" and "Gradient Descent, How Neural Networks Learn" on YouTube (15 min each).

Neural Networks & Deep Learning

A neural network is a function `f(x, W)` that maps input `x` (token IDs) to output `y` (label probabilities) using learned parameters `W` (weights). Training means finding weights that minimize the error on training data.

- **Forward pass:** input flows through layers (embeddings → attention → feed-forward → classification head) and produces a prediction.
- **Loss function:** cross-entropy loss compares the prediction to the correct label. Lower loss = better fit.
- **Backward pass (backpropagation):** the loss signal flows backward through the network, computing gradients for every weight.
- **Optimizer:** AdamW (the standard for Transformers) uses gradients to update weights. It maintains per-parameter learning rates and momentum.
- **Epoch:** one full pass through the entire training dataset (392k pairs for XNLI Hindi). Multiple epochs let the model see each example more than once, but too many cause overfitting.
- **Batch:** a group of `BATCH_SIZE` examples processed together. Larger batches give smoother gradients but use more GPU memory.
- **Overfitting:** when the model memorizes training data but fails on unseen data. Prevented by: stopping early, using a validation set, regularization, and dropout (built into XLM-RoBERTa).

The Transformer Architecture

This is the most important section. XLM-RoBERTa is a Transformer, and you must understand its components to interpret training output and write the paper.

4.4.1 Embedding Layer

Every token ID (e.g., 4823 for `kar` or 104 for the subword token `ing`) is converted to a dense vector (**embedding**) of size 768 (for `xlm-roberta-base`). These embeddings are learned: semantically similar tokens have similar vectors. Three separate embeddings are summed per token: token embedding + position embedding (tells the model where each token is in the sequence) + language embedding (tells XLM-RoBERTa which language the input is in; crucial for multilingual models).

4.4.2 Self-Attention Mechanism

This is the heart of the Transformer. For each token, self-attention computes a weighted sum of all other tokens' representations. The weight (*attention score*) between token A and token B measures how relevant B is to understanding A.

- Each token produces three vectors: **Query (Q)**, **Key (K)**, **Value (V)** via learned weight matrices.
- Attention score between token i and j = softmax $\left(\frac{Q_i \cdot K_j}{\sqrt{d_k}}\right)$. The dot product measures similarity; softmax normalizes all scores to sum to 1; dividing by $\sqrt{d_k}$ prevents extreme values.
- The output for token i is the weighted sum of all Value vectors: $\sum_j \text{score}_{i,j} V_j$.
- This means every token can directly "look at" every other token in one step — unlike RNNs that process left-to-right and lose distant context.

4.4.3 Multi-Head Attention

Instead of one attention calculation, XLM-RoBERTa runs 12 (base) or 16 (large) **heads** in parallel. Each head learns to focus on different relationships: one head might track subject-verb agreement, another might track negation words, another might track adjective-noun pairs. The 12 outputs are concatenated and projected back to 768 dimensions.

4.4.4 Transformer Block Structure

A single Transformer block (layer) does:

1. **Multi-head self-attention** on the input
2. **Add & LayerNorm:** add the attention output to the original input (residual connection) then normalize. Residual connections help gradients flow through deep networks.
3. **Feed-forward network (FFN):** two linear layers with a GELU activation in between. This is where the model does most of its "thinking" — each token's representation is transformed non-linearly.
4. **Add & LayerNorm:** another residual connection and normalization.

XLM-RoBERTa-base stacks 12 such blocks. Each block gives the model a chance to refine its representation based on wider context.

4.4.5 Classification Head

After all 12 layers, the model outputs a vector of size 768 for each token. For sequence classification, we take the first token (a special `<s>` token, also called CLS) and pass it through a new linear layer that maps $768 \to 3$. Those 3 numbers are the *logits* for Entailment, Neutral, and Contradiction. Softmax converts them to probabilities.

4.4.6 XLM-RoBERTa vs. BERT

XLM-RoBERTa is an improved multilingual BERT:

- **More data:** trained on 2.5 TB of CommonCrawl text across 100 languages (vs. BERT's English-only 16 GB).
- **No language ID required:** the model infers the input language from the tokens themselves.
- **SentencePiece tokenizer:** subword tokenizer that handles any language with a 250k vocabulary (vs. BERT's 30k English WordPiece). A word like "**namaste**" might be split into "**nam**" + "**ste**" if the whole word isn't in the vocabulary.
- **Dynamic masking:** BERT's masking pattern was fixed during pre-processing; RoBERTa (and XLM-R) re-generates masks each time data is seen, leading to more robust learning.
- **Larger batch and more steps:** trained for 500k steps with large batches vs. BERT's 1M steps with smaller batches.

Tokenization in Detail

Tokenization converts raw text (Hindi sentences) into integers the model can process.

- **SentencePiece:** XLM-RoBERTa uses a 250k-token unigram language model tokenizer. It learns a probability distribution over subword units and splits unknown words into the most likely sequence of known subwords.

- **Padding & truncation:** all sequences in a batch must be the same length. Shorter sequences get `<pad>` tokens (ID = 1) added to the end (padding). Sequences longer than `max_length` (128) get cut (truncation).
- **Attention mask:** a binary mask (1 = real token, 0 = padding token) tells the model which positions to attend to and which to ignore.
- **Output format:** the tokenizer returns a dict with `input_ids` (the integer IDs), `attention_mask` (the 0/1 mask), and nothing else (since XLM-R doesn't use token type IDs).

Transfer Learning & Fine-Tuning

This is the single most important concept for the entire project.

- **Pre-training:** XLM-RoBERTa was trained on a massive corpus (2.5 TB) to predict masked tokens (like a fill-in-the-blank game). It learned general language understanding — grammar, syntax, some semantics, cross-lingual alignment — without any labeled data.
- **Fine-tuning:** we take the pre-trained model and continue training on a specific task (NLI) with labeled data. The pre-trained weights are a great starting point; fine-tuning adapts them to our task with relatively little data (392k pairs is small for a 278M-parameter model).
- **Why this works:** The general linguistic knowledge from pre-training transfers to the specific task. The classification head (randomly initialized at first) quickly learns to interpret the 768-dimensional representations for entailment. Without pre-training, 392k pairs would be far too little data to train a 278M-parameter model from scratch.
- **Learning rate:** fine-tuning uses a much smaller learning rate (2e-5) than pre-training (1e-4). We want to adapt, not overwrite, the pre-trained knowledge.
- **Warmup:** the learning rate starts at 0 and linearly increases to 2e-5 over the first 10% of training steps. This prevents the randomly initialized classification head from causing large gradient updates that would disrupt the pre-trained weights.

The Textual Entailment (NLI) Task

4.7.1 Task Structure

Premise: A statement of fact (reference sentence).
Hypothesis: A claim to be checked against the premise.
Label: one of three:

- **Entailment (0):** the hypothesis follows logically from the premise.
- **Neutral (1):** the hypothesis might be true or false given the premise — no logical connection.
- **Contradiction (2):** the hypothesis is false given the premise.

Example (from XNLI Hindi):

- Premise: "**ladka kamre mein khel raha hai**" (The boy is playing in the room)
- Hypothesis: "**ladka ghar ke andar hai**" (The boy is inside the house)
- Label: Entailment

4.7.2 Why 3 classes instead of 2

The original paper (Khader et al., 2016) used binary classification: Entails / Not Entails. Three-class NLI is more realistic — the presence or absence of evidence is not the same as contradiction. A model that only distinguishes "entails vs. not-entails" cannot tell the difference between "unrelated" and "actively contradicts."

PyTorch for This Project

You don't need to be a PyTorch expert, but you should understand these specific things:

- **Tensors:** the fundamental data structure. A tokenized batch is a `torch.LongTensor` of shape `[16, 128]` (16 sentences, each 128 token IDs). The model's hidden states are `torch.FloatTensor` of shape `[16, 128, 768]`.
- **.to(device):** moves tensors and models between CPU, MPS (Apple GPU), or CUDA (NVIDIA GPU). Mismatched device = crash:

```
model = model.to("mps")
inputs = inputs.to("mps")
```

- **torch.no_grad():** disables gradient computation during evaluation. Saves memory and is 2-3× faster.
- **loss.backward():** computes gradients. Only called during training, not evaluation.
- **optimizer.step():** updates weights using gradients. Combined with `loss.backward()`, this is the training loop's core.
- **MPS backend (Apple Silicon):** Apple's Metal Performance Shaders for GPU acceleration. Not as mature as CUDA, but works for FP32 training. Some operations fall back to CPU silently.

Hugging Face Ecosystem

The Hugging Face libraries do the heavy lifting so you don't have to write transformers from scratch.

- **AutoTokenizer:** loads the correct tokenizer for any model by name. Handles padding, truncation, special tokens, and device placement. Returns pre-formatted dicts ready for the model.
- **AutoModelForSequenceClassification:** loads a pre-trained model with a classification head. The `num_labels=3` argument replaces the default head with a 3-output linear layer. The warning `"Some weights were not initialized"` is expected — it refers to the randomly initialized head.
- **Datasets library:** loads datasets from Hugging Face Hub or local files. Uses Apache Arrow under the hood (fast, memory-mapped). Supports `.map()` for efficient batch processing and `.select()` for subsetting.
- **Trainer API:** the highest-level abstraction. Wraps the training loop (forward, backward, optimizer step, logging, evaluation, checkpointing). You configure it with `TrainingArguments` and call `trainer.train()`. Beneath it, the loop does:

```
for epoch in range(num_epochs):
    for batch in dataloader:
        outputs = model(**batch)
```

```
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()
```

- **TrainingArguments:** the configuration object. Key fields: `output_dir`, `num_train_epochs`, `per_device_train_batch_size`, `logging_strategy`, `eval_strategy`, `save_strategy`, `learning_rate`, `warmup_ratio`.

Evaluation Metrics

4.10.1 Accuracy

The fraction of predictions that exactly match the correct label. This is the main metric we report, but it can be misleading if classes are imbalanced (XNLI Hindi is balanced at 33% each).

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}}$$

4.10.2 Precision, Recall, F1

For each class (e.g., Entailment):

- **Precision:** of all examples the model labeled as Entailment, what fraction were actually Entailment? High precision = few false positives.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall:** of all examples that are truly Entailment, what fraction did the model catch? High recall = few false negatives.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- **F1 Score:** harmonic mean of precision and recall. A balanced measure.

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

XNLI evaluation typically reports **macro F1** (average F1 across all three classes, treating each class equally) alongside accuracy.

4.10.3 Confusion Matrix

A 3×3 table showing predicted vs. actual labels for every test example. Rows = actual labels, columns = predicted labels. The diagonal shows correct predictions; off-diagonal numbers show specific error patterns (e.g., confusing Neutral for Contradiction). The confusion matrix is the most informative single visualization.

Hardware Awareness

Understanding your hardware constraints changes how you train:

- **Unified memory (M5):** the CPU and GPU share 24 GB RAM. No separate VRAM, but this also means the GPU doesn't have its own dedicated high-bandwidth memory. MPS uses the same pool.
- **Paging:** with 24 GB unified memory, a 278M-parameter model in FP32 (∼1.1 GB) plus 392k tokenized sequences (∼2 GB) fits easily. However, PyTorch's MPS backend may page to swap if memory pressure is high.
- **FP32:** we use float32 precision (not float16 mixed precision). MPS mixed precision (AMP) is experimental on Apple Silicon and can cause crashes. FP32 is slower but stable.
- **Batch size trade-off:** larger batches = faster training per-sample but require more memory. Batch size 16 is a conservative choice for 24 GB.
- **Training time estimate:** at ∼1.07 samples/second on M5, one epoch of 392k samples takes ∼102 hours÷3600 = approx. 6.1 hours. Each additional epoch adds roughly the same time.

Book Reading Plan: "Fundamentals of Deep Learning" (Buduma & Locascio)

This is the primary textbook for the course. Read **selected sections only** — the full book is 360+ pages and much of it is not needed for this project.

Read?	Ch. / Section	Why
✓	Ch 3: The Neural Network	Activations (sigmoid, tanh, ReLU), softmax output, feed-forward structure
✓	Ch 4: Training NN	Backpropagation, gradient descent, overfitting, validation sets
✓	Ch 5: PyTorch (all)	Tensors, gradients, nn.Module, Datasets, DataLoaders
✓	Ch 6: *Adam, Momentum, RMSProp only*	Skip second-order methods; read just the optimizer sections
✓	Ch 8: *Word2Vec & embeddings concept only*	Skip autoencoders, PCA; understand embedding layers
✓	Ch 9: *Self-Attention & Transformers only (p. 239)*	Skip RNNs, LSTMs, N-grams, POS tagging
○	Ch 2: *Cross entropy / KL divergence only*	Read just that section if cross-entropy is unfamiliar
○	Ch 1: Linear Algebra	Reference only — skim if comfortable with matrices
—	Ch 7: Convolutional Neural Networks	Not used in NLP transformers
—	Ch 10: Generative Models	GANs / VAEs not relevant
—	Ch 11: Interpretability	LIME / SHAP not needed
—	Ch 12: Memory Networks	NTM / DNC not used
—	Ch 13: Reinforcement Learning	Entirely separate topic

Estimated reading: ∼140 pages across Ch 3, 4, 5, 6 (partial), 8 (partial), 9 (partial). Read in this order: 3 → 4 → 5 → 6 (optimizers) → 9 (transformer) → 8 (embeddings).

Additional Learning Resources

1. **3Blue1Brown** — "Neural Networks" series; "Attention in Transformers, Visually Explained" (YouTube, 15-30 min each)
2. **PyTorch in 60 Minutes** https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html
3. **Hugging Face NLP Course (Chapters 1-3)** https://huggingface.co/learn/nlp-course
4. **Illustrated Transformer (Jay Alammar)** http://jalammar.github.io/illustrated-transformer/
5. **The Annotated Transformer (harvardnlp)** http://nlp.seas.harvard.edu/2018/04/03/attention.html
6. **Text Classification with Transformers** https://huggingface.co/docs/transformers/tasks/sequence_classification
7. **XNLI Paper** — Conneau et al., 2018, "XNLI: Evaluating Cross-lingual Sentence Representations"
8. **XLM-RoBERTa Paper** — Conneau et al., 2019, "Unsupervised Cross-lingual Representation Learning at Scale"

Technical Stack

Component	Choice
Language	Python 3.10+
Deep Learning Framework	PyTorch 2.0+ (MPS backend)
Hugging Face Transformers	Model loading, tokenization, training
Hugging Face Datasets	XNLI Hindi dataset loading
Model	XLM-RoBERTa-base (125M params)
Hardware	MacBook Air M5 (10-core Apple GPU, 24GB RAM)
Development	VS Code
Version Control	Git

Part I

Phase 1: NLI Assignment

Implementation Plan

Phase 1: Environment Setup (Day 1)

1. Create a Python virtual environment
2. Install dependencies:

```
pip install torch transformers datasets evaluate scikit-learn matplotlib
```

3. Verify GPU access:

```
python -c "import torch; print(torch.backends.mps.is_available())"
```

Phase 2: Dataset Exploration (Day 2)

1. Load XNLI for Hindi:

```
from datasets import load_dataset
dataset = load_dataset("xnli", "hi")
print(dataset["train"][0])
```

2. Understand label mapping: 0 = Entailment, 1 = Neutral, 2 = Contradiction
3. Statistics: 392k training pairs (machine-translated from English), 2.5k validation, 5k test

Phase 3: Model & Tokenizer Setup (Day 3)

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

model_name = "xlm-roberta-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=3
)
```

Phase 4: Training Pipeline (Day 4-5)

1. Tokenize the dataset (padding, truncation to 128 tokens)
2. Set device to Metal (MPS) for Apple Silicon:

```
import torch
device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
model.to(device)
```

3. Configure TrainingArguments:

```
TrainingArguments(
    output_dir="./xnli-hindi-model",
    evaluation_strategy="epoch",
    save_strategy="epoch",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=64,
    warmup_ratio=0.1,
    logging_dir="./logs",
    fp16=True,
```

```
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
)
```

4. Define compute_metrics function (accuracy, precision, recall, F1)
5. Run Trainer:

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=encoded["train"],
    eval_dataset=encoded["validation"],
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)
trainer.train()
```

6. Expected training time: 30-45 minutes on M5 Apple GPU (per epoch)

Phase 5: Evaluation (Day 6)

```
results = trainer.evaluate(encoded["test"])
print("Test Accuracy:", results["eval_accuracy"])
```

Expected results:

- **Accuracy:** 85-88%
- **F1 Score:** 0.84-0.87
- **Precision:** 0.83-0.87
- **Recall:** 0.85-0.88

Phase 6: Inference Demo (Day 7)

Build a reusable inference function:

```
device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
model.to(device)

def predict_entailment(premise, hypothesis):
    inputs = tokenizer(premise, hypothesis,
                       return_tensors="pt", truncation=True)
    inputs = {k: v.to(device) for k, v in inputs.items()}
    outputs = model(**inputs)
    probs = torch.nn.functional.softmax(outputs.logits, dim=-1)
    label = torch.argmax(probs).item()
    return {0: "Entailment", 1: "Neutral", 2: "Contradiction"}[label]
```

Expected Results & Comparison

Table 2: Comparison: Original Work vs. Proposed Work

Metric	**Original (Arabic)**	**Proposed (Hindi)**
Language	Arabic	Hindi
Dataset	ArbTEDS (618 pairs)	XNLI Hindi (392k train, 5k test)
Approach	Lexical + bigram matching	XLM-RoBERTa fine-tuning
Labels	Binary (Entails / Not)	3-way (Entailment / Neutral / Contradiction)
Algorithm	Threshold-based rules	12-layer Transformer with attention
Feature Engineering	Manual (POS, NER, stems)	None (learned end-to-end)
Accuracy	61% (precision/recall)	Expected 85-88%
Training Time	N/A (rule-based)	30-45 min per epoch on M5 GPU

Part II

Phase 2: KG Survey Paper

Overview

Write a technical review paper on **Knowledge Graph Construction for Hindi / Indic Languages**, modeled after the Arabic KG survey (Ahmed et al., 2022). This will be a supervised undergraduate publication with guidance from the course professor.

Why a Review Paper?

- No novel implementation required — pure reading, synthesis, and writing
- Follow the exact structure of the Arabic KG survey (Ahmed et al., 2022) as a template
- Manageable scope: Indic KG literature is smaller than Arabic, requiring ∼40 papers to read
- Builds academic writing skills for future publications
- Suitable for a first undergraduate paper regardless of journal quartile (Q1–Q4)

Template Paper

The paper *"Arabic Knowledge Graph Construction: A close look in the present and into the future"* (Ahmed et al., 2022, Journal of King Saud University – Computer and Information Sciences) provides the template structure:

1. **Introduction** — Motivation for KGs, why Indic languages need dedicated KG research
2. **Indic Languages Overview** — Linguistic challenges: morphology, script diversity, code-mixing, resource scarcity
3. **Knowledge Graph Overview** — Definition, construction phases (NER, RE, entity linking, fusion), applications (QA, recommendation)
4. **Existing Approaches** — Survey of existing KG systems for Indic languages (Hindi, Urdu, Bengali, Tamil, etc.) and cross-lingual approaches
5. **Challenges & Problems** — Data sparsity, tool availability, annotation quality, evaluation benchmarks
6. **Future Directions** — LLM-based KG construction, cross-lingual alignment, Indic Wikipedia mining
7. **Conclusion**

Research Methodology

1. Search major venues (ACL, EMNLP, NAACL, COLING, LREC, TALLIP) for Indic KG papers using keywords: "Hindi knowledge graph," "Indic knowledge graph construction," "Urdu KG," "Indo-Aryan KG"
2. Categorize papers by: language covered, approach (rule-based / ML / hybrid), construction phase (NER / RE / entity linking / fusion), application domain

3. Identify gaps and open problems (e.g., no large-scale Hindi KG benchmark, limited Urdu Wikidata coverage)
4. Synthesize into a taxonomy table (similar to Ahmed et al. Fig. 4)

Expected Timeline

Week	Activity
4	Read Arabic KG survey as template; search and collect 40+ Indic KG papers
5	Read and annotate papers; categorize by approach and language
6	Write first draft following template structure
7	Iterate with professor feedback; revise
8	Polish formatting, finalize references, submit

Target Venues

Suitable venues for an undergraduate survey paper:

- ACL Student Research Workshop (ACL-SRW)
- Workshop on NLP for Similar Languages, Varieties and Dialects (VarDial)
- Conference on Empirical Methods in NLP Student Workshop
- Journal of King Saud University – Computer and Information Sciences (same venue as template)
- Any Q3/Q4 journal recommended by professor

Paper Structure (Phase 1: NLI Assignment)

The assignment paper will follow this structure (modeled after the original paper's format):

1. **Abstract** — 150-200 words summarizing the adaptation
2. **Introduction** — Textual Entailment task, motivation for Hindi, gaps in original work
3. **Related Work** — Original paper, XLM-RoBERTa, XNLI, cross-lingual NLI
4. **Dataset** — XNLI Hindi: size, sources, label distribution, sample pairs
5. **Methodology** — XLM-RoBERTa architecture, fine-tuning procedure, hyperparameters
6. **Experiments & Results** — Accuracy, precision, recall, F1; comparison table with original
7. **Conclusion & Future Work** — Summary, limitations, directions (dialectal Hindi, other Indic languages)
8. **References** — 6-8 key citations

Paper Structure (Phase 2: KG Survey)

See Section 8 for the full outline of the KG review paper.

Timeline (8 Weeks)

Phase 1: NLI Assignment (Weeks 1–3)

Week	Activity
1	Environment setup; PyTorch basics; Hugging Face course (Ch 1-3); explore XNLI
2	Read prerequisites (Ch 3, 4, 5 from textbook); write tokenization and training script
3	Run training; evaluate on test set; build inference demo; write NLI assignment paper

Phase 2: KG Survey Paper (Weeks 4–8)

Week	Activity
4	Read Arabic KG survey (Ahmed et al., 2022) as template; collect 40+ Indic KG papers
5	Read and annotate papers; categorize by approach, language, and construction phase
6	Write first draft following template structure
7	Iterate with professor feedback; revise
8	Polish, format for target venue, submit

Deliverables

Phase 1 (NLI Assignment)

1. **Training script** (`train.py`) — End-to-end fine-tuning pipeline
2. **Evaluation script** (`evaluate.py`) — Metrics and visualization
3. **Inference script** (`inference.py`) — Demo with sample Hindi pairs
4. **Metrics report** (`results/metrics.json`) — Accuracy, precision, recall, F1
5. **Confusion matrix** (`results/confusion_matrix.png`)
6. **Assignment paper** (`nli-paper.pdf`) — Full write-up in LaTeX/Word
7. **README** — Setup and usage instructions

Phase 2 (KG Survey Paper)

1. **Review paper** (`kg-survey-paper.pdf`) — Technical review on KG construction for Indic languages
2. **Annotated bibliography** (`bibliography.bib`) — All referenced papers
3. **Literature matrix** (`lit-matrix.xlsx`) — Categorized summary of each paper read

References

1. Khader, M., Awajan, A., & Alkouz, A. (2016). Textual Entailment for Arabic Language Based on Lexical and Semantic Matching. *International Journal of Computing & Information Sciences*, 12(1), 67-74.
2. Conneau, A., et al. (2019). Unsupervised Cross-lingual Representation Learning at Scale. *arXiv preprint arXiv:1911.02116*.
3. Conneau, A., et al. (2018). XNLI: Evaluating Cross-lingual Sentence Representations. *Proceedings of EMNLP 2018*.
4. Devlin, J., et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL 2019*.
5. Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. *Proceedings of EMNLP 2020: System Demonstrations*.
6. Alabbas, M. (2013). A Dataset for Arabic Textual Entailment. *Proceedings of the Student Research Workshop associated with RANLP 2013*.
7. Ahmed, I. A., et al. (2022). Arabic Knowledge Graph Construction: A close look in the present and into the future. *Journal of King Saud University – Computer and Information Sciences*, 34(9), 6505-6523.

End of Plan